

**ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ
KHOA CÔNG NGHỆ THÔNG TIN**



**ĐỒ ÁN MÔN HỌC: CHUYÊN ĐỀ ASP.NET
HỌC KỲ III, NĂM HỌC 2025-2026**

**XÂY DỰNG
WEBSITE NGHE NHẠC TRỰC TUYẾN**

Giảng viên hướng dẫn:

THS. Đoàn Phước Miên

Sinh viên thực hiện:

Họ tên: Nguyễn Việt Hiếu

MSSV:170124891

Lớp:DK24TT80171

Vĩnh long, năm 2026

Trong suốt quá trình học tập và thực hiện đề án môn học Chuyên đề ASP.NET, tôi đã nhận được rất nhiều sự hỗ trợ, giúp đỡ từ phía nhà trường, gia đình và bạn bè.

Đầu tiên, tôi xin gửi lời cảm ơn chân thành đến Trường Đại học Trà Vinh đã tạo điều kiện thuận lợi về cơ sở vật chất, trang thiết bị phục vụ học tập và nghiên cứu. Tôi xin bày tỏ lòng biết ơn sâu sắc đến quý thầy cô Khoa Kỹ thuật và Công nghệ đã tận tình giảng dạy và truyền đạt kiến thức trong suốt quá trình học tập.

Đặc biệt, tôi xin gửi lời cảm ơn trân trọng đến giảng viên hướng dẫn đã dành thời gian định hướng, góp ý và chỉnh sửa đề án từ những bước đầu tiên đến khi hoàn thành.

Cuối cùng, tôi xin cảm ơn gia đình và bạn bè đã luôn ở bên động viên, hỗ trợ tôi trong suốt quá trình thực hiện đề án.

Mặc dù đã cố gắng hết sức, nhưng đề án không tránh khỏi những thiếu sót. Tôi rất mong nhận được sự góp ý, nhận xét từ quý thầy cô để đề án được hoàn thiện hơn.

Vĩnh Long, ngày tháng năm 2026

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

MỤC LỤC

CHƯƠNG 1: TỔNG QUAN	3
1.1. Tổng quan về lĩnh vực	3
1.2. Các hệ thống tương tự	3
1.3. Yêu cầu bài toán	3
CHƯƠNG 2: NGHIÊN CỨU LÝ THUYẾT	5
2.1. ASP.NET Core 8	5
2.1.1. Tổng quan ASP.NET Core	5
2.1.2. Kiến trúc Clean Architecture.....	5
2.1.3. Entity Framework Core	5
2.1.4. JWT Authentication.....	6
2.2. SQL Server và Cơ sở dữ liệu.....	6
2.2.1. SQL Server 2022	6
2.2.2. Docker cho SQL Server.....	6
2.3. Next.js và Giao diện Frontend.....	6
2.3.1. Next.js 14 (App Router)	6
2.3.2. Zustand — State Management	7
2.3.3. Tailwind CSS.....	7
2.4. Các công nghệ và thư viện khác	7
2.4.1. AutoMapper.....	7
2.4.2. FluentValidation	8
2.4.3. BCrypt.Net.....	8
2.4.4. HTTP Range Requests	8
CHƯƠNG 3: HIỆN THỰC HÓA NGHIÊN CỨU	9
3.1. Mô tả bài toán	9
3.2. Yêu cầu chức năng	9
3.3. Mô hình cơ sở dữ liệu.....	10
.....	Error! Bookmark not defined.
3.4. Sơ đồ Use Case.....	14
3.5. Kiến trúc hệ thống	15
3.6. Thiết kế API.....	16
CHƯƠNG 4: KẾT QUẢ NGHIÊN CỨU	18
4.1. Môi trường chạy ứng dụng	18
4.2. Giao diện trang chủ	18

4.3. Giao diện Đăng nhập / Đăng ký	18
4.4. Giao diện Khám phá nhạc	19
4.5. Trình phát nhạc (PlayerBar)	20
4.6. Giao diện Upload bài hát.....	21
4.7. Admin Dashboard.....	22
4.8. Quản lý bài hát chờ duyệt.....	23
4.9. Swagger API Documentation	24
4.10. Đánh giá kết quả.....	24
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	26
5.1. Kết luận.....	26
5.2. Hướng phát triển.....	26
DANH MỤC TÀI LIỆU THAM KHẢO	28
PHỤ LỤC	29
Phụ lục A: Hướng dẫn cài đặt và chạy ứng dụng	29
Phụ lục B: Cấu trúc thư mục dự án	30

DANH MỤC HÌNH ẢNH

Hình 1 Mô hình cơ sở dữ liệu (ERD)10

Hình 2 Sơ đồ Use Case14

Hình 3 Kiến trúc hệ thống15

Hình 4 Giao diện trang chủ18

Hình 5 Giao diện trang đăng nhập.....19

Hình 6 Giao diện Khám phá nhạc19

Hình 7 Trình phát nhạc (PlayerBar)20

Hình 8 Giao diện upload bài hát.....21

Hình 9 Admin Dashboard.....22

Hình 10 Quản lý bài hát chờ duyệt.....23

Hình 11 Swagger API Documentation24

DANH MỤC BẢNG BIỂU

Bảng 1 Chức năng hệ thống9

Bảng 2 Mô tả bảng Users10

Bảng 3 Mô tả bảng Songs.....11

Bảng 4 Mô tả bảng Artists.....11

Bảng 5 Mô tả bảng Albums.....12

Bảng 6 Mô tả bảng Playlists12

Bảng 7 Mô tả bảng PlaylistSongs.....12

Bảng 8 Mô tả bảng FavoriteSongs13

Bảng 9 Mô tả bảng PlayHistories13

Bảng 10 Danh sách API Endpoints chính17

Bảng 11 Tài khoản mẫu được seed sẵn17

Bảng 12 Đánh giá kết quả25

TÓM TẮT ĐỒ ÁN

EMusic là một ứng dụng web nghe nhạc trực tuyến được xây dựng với kiến trúc RESTful API sử dụng ASP.NET Core 8 làm backend và Next.js 14 làm frontend. Hệ thống cho phép người dùng nghe nhạc trực tuyến, quản lý playlist cá nhân, đánh dấu bài hát yêu thích và upload nhạc lên nền tảng.

Về mặt kỹ thuật, backend được thiết kế theo kiến trúc Clean Architecture với 3 tầng rõ ràng (API, Core, Infrastructure), sử dụng Entity Framework Core 8 để tương tác với SQL Server, JWT Bearer Authentication để bảo mật và AutoMapper để ánh xạ dữ liệu. Hệ thống phân quyền 3 cấp (User, UserPro, Admin) đảm bảo kiểm soát truy cập phù hợp.

Frontend được xây dựng với Next.js 14 (App Router), Tailwind CSS theo phong cách thiết kế Spotify, Zustand để quản lý trạng thái ứng dụng. Trình phát nhạc toàn cục hỗ trợ phát/dừng, tua, điều chỉnh âm lượng, shuffle và repeat.

Kết quả đạt được bao gồm: hệ thống backend với 8 controllers và 35+ endpoints, frontend với 10 trang chức năng, khả năng stream audio với HTTP Range Request (hỗ trợ tua nhạc), và trang Admin Dashboard quản lý toàn bộ hệ thống.

MỞ ĐẦU

1. Lý do chọn đề tài

Trong thời đại công nghệ số, âm nhạc trực tuyến đã trở thành một phần không thể thiếu trong cuộc sống hàng ngày. Các nền tảng như Spotify, Apple Music hay Zing MP3 đã chứng minh rằng việc nghe nhạc qua internet không chỉ tiện lợi mà còn mang lại trải nghiệm phong phú cho người dùng với hàng triệu bài hát chỉ cần một cú nhấp chuột.

Môn học Chuyên đề ASP.NET cung cấp nền tảng lý thuyết vững chắc về phát triển web với nền tảng .NET. Để ứng dụng và củng cố kiến thức đã học, tôi chọn đề tài xây dựng website nghe nhạc trực tuyến EMusic — một ứng dụng thực tế kết hợp nhiều kỹ thuật quan trọng như REST API, xác thực JWT, upload và stream file, quản lý quyền người dùng và thiết kế giao diện hiện đại.

2. Mục tiêu nghiên cứu

- Xây dựng hệ thống backend hoàn chỉnh với ASP.NET Core 8 theo kiến trúc Clean Architecture
- Thiết kế RESTful API bao gồm xác thực, phân quyền và quản lý tài nguyên
- Xây dựng giao diện người dùng hiện đại với Next.js và Tailwind CSS theo phong cách Spotify
- Cài đặt tính năng stream audio hỗ trợ HTTP Range Request
- Xây dựng hệ thống phân quyền 3 cấp: User, UserPro, Admin
- Triển khai cơ sở dữ liệu bằng Docker cho SQL Server

3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu: Công nghệ ASP.NET Core 8, Entity Framework Core, JWT Authentication, Next.js 14, SQL Server.

Phạm vi: Xây dựng ứng dụng web nghe nhạc với các chức năng cốt lõi bao gồm xác thực người dùng, phát nhạc trực tuyến, quản lý playlist, yêu thích bài hát, upload nhạc và quản trị hệ thống. Ứng dụng chạy trên môi trường local, không triển khai lên server thực tế.

4. Phương pháp nghiên cứu

- Nghiên cứu tài liệu chính thức của Microsoft về ASP.NET Core, Entity Framework Core
- Tham khảo tài liệu Next.js, Tailwind CSS, Zustand
- Áp dụng phương pháp phát triển theo từng giai đoạn: Foundation → Core Domain → Playlists & Player UI → Admin Dashboard
- Kiểm thử thủ công toàn bộ chức năng qua Swagger UI và giao diện web

CHƯƠNG 1: TỔNG QUAN

1.1. Tổng quan về lĩnh vực

Ngành công nghiệp âm nhạc số đang phát triển mạnh mẽ với doanh thu toàn cầu vượt 17 tỷ USD năm 2023 theo IFPI. Các nền tảng streaming chiếm hơn 67% doanh thu âm nhạc toàn cầu, với Spotify dẫn đầu bằng 600 triệu người dùng tích cực.

Trong nước, các nền tảng như Zing MP3, NhacCuaTui, Keeng đã phục vụ hàng chục triệu người dùng Việt Nam. Điều này cho thấy nhu cầu nghe nhạc trực tuyến là rất lớn và đây là lĩnh vực ứng dụng phù hợp để học tập và nghiên cứu về phát triển web hiện đại.

1.2. Các hệ thống tương tự

Một số hệ thống nghe nhạc trực tuyến phổ biến được nghiên cứu để tham khảo:

- **Spotify:** Nền tảng streaming hàng đầu thế giới với giao diện tối, thiết kế tối giản, PlayerBar toàn cục không bị gián đoạn khi chuyển trang. EMusic lấy cảm hứng thiết kế từ Spotify.

- **Zing MP3:** Nền tảng Việt Nam hỗ trợ upload nhạc theo cơ chế kiểm duyệt, phân chia loại tài khoản (free/premium).

- **SoundCloud:** Cho phép người dùng upload và chia sẻ nhạc tự do với cơ chế phê duyệt.

EMusic học hỏi từ các hệ thống trên và tập trung vào việc xây dựng kiến trúc backend rõ ràng, sạch sẽ phù hợp với mục đích học tập.

1.3. Yêu cầu bài toán

Yêu cầu chức năng:

- Người dùng có thể đăng ký, đăng nhập và quản lý tài khoản
- Nghe nhạc trực tuyến với khả năng tua, điều chỉnh âm lượng
- Tìm kiếm bài hát, nghệ sĩ
- Quản lý playlist cá nhân
- Đánh dấu bài hát yêu thích

- Upload bài hát (có kiểm duyệt với tài khoản thường)
- Admin quản lý toàn bộ hệ thống

Yêu cầu phi chức năng:

- API theo chuẩn RESTful
- Bảo mật bằng JWT Authentication
- Hỗ trợ stream audio với HTTP Range Request
- Giao diện responsive, dark theme theo phong cách Spotify
- Dữ liệu mẫu được seed tự động khi khởi động

CHƯƠNG 2: NGHIÊN CỨU LÝ THUYẾT

2.1. ASP.NET Core 8

2.1.1. Tổng quan ASP.NET Core

ASP.NET Core là framework web mã nguồn mở, đa nền tảng của Microsoft, được phát triển từ năm 2016 như là phiên bản viết lại hoàn toàn của ASP.NET. Phiên bản 8 (LTS – Long Term Support) phát hành tháng 11/2023 mang đến nhiều cải tiến về hiệu năng và tính năng.

Các đặc điểm chính của ASP.NET Core 8:

- **Đa nền tảng:** Chạy trên Windows, Linux, macOS
- **Hiệu năng cao:** Consistently ranked top trong TechEmpower benchmarks
- **Dependency Injection tích hợp sẵn:** IoC container built-in
- **Minimal API và Controller-based API:** Hai mô hình phát triển linh hoạt
- **Middleware pipeline:** Cơ chế xử lý request/response theo chuỗi

2.1.2. Kiến trúc Clean Architecture

Clean Architecture (kiến trúc sạch) do Robert C. Martin đề xuất, tách biệt logic nghiệp vụ khỏi infrastructure và UI. EMusic áp dụng kiến trúc này với 3 tầng:

- **MusicApp.Core:** Chứa Entities (thực thể), DTOs (Data Transfer Objects), Interfaces và Enums. Không phụ thuộc vào bất kỳ thư viện nào ngoài .NET runtime.
- **MusicApp.Infrastructure:** Triển khai các interface từ Core, bao gồm ApplicationDbContext, Repositories, FileService, TokenService. Phụ thuộc vào Core.
- **MusicApp.API:** Tầng trình bày, chứa Controllers, Middleware, Mappings. Phụ thuộc vào cả Core và Infrastructure.

2.1.3. Entity Framework Core

Entity Framework Core (EF Core) là ORM (Object-Relational Mapper) chính thức của Microsoft cho .NET. EMusic sử dụng EF Core 8 với các tính năng:

- **Code First:** Định nghĩa model bằng C# class, EF Core tạo ra database schema

- **Migrations:** Quản lý thay đổi schema theo thời gian
- **LINQ queries:** Truy vấn dữ liệu bằng C# thay vì SQL thuần
- **Navigation properties:** Quan hệ giữa các bảng được biểu diễn qua thuộc tính

2.1.4. JWT Authentication

JSON Web Token (JWT) là chuẩn mở (RFC 7519) để truyền thông tin bảo mật giữa các bên dưới dạng JSON object được ký số. Cấu trúc gồm 3 phần: Header.Payload.Signature.

EMusic sử dụng JWT Bearer Authentication với các đặc điểm:

- Token được tạo khi đăng nhập thành công bằng HMAC-SHA256
- Token chứa UserId, Username, Role trong Payload
- Có thời hạn 7 ngày
- Frontend lưu token trong localStorage và tự động đính kèm vào header mỗi request

2.2. SQL Server và Cơ sở dữ liệu

2.2.1. SQL Server 2022

Microsoft SQL Server là hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) được sử dụng rộng rãi trong môi trường doanh nghiệp. EMusic sử dụng SQL Server 2022 chạy trong Docker container để đơn giản hóa việc cài đặt môi trường.

2.2.2. Docker cho SQL Server

Docker container cho phép chạy SQL Server mà không cần cài đặt trực tiếp trên máy host. File docker-compose.yml định nghĩa service sqlserver với image mcr.microsoft.com/mssql/server:2022-latest, mở port 1433 và thiết lập biến môi trường SA_PASSWORD và ACCEPT_EULA.

2.3. Next.js và Giao diện Frontend

2.3.1. Next.js 14 (App Router)

Next.js là framework React được phát triển bởi Vercel, nổi tiếng với Server-Side Rendering (SSR) và Static Site Generation (SSG). Phiên bản 14 giới thiệu App Router — mô hình routing mới dựa trên file system với hỗ trợ Server Components và layouts.

EMusic sử dụng App Router với các tính năng:

- **Layouts lồng nhau:** Root layout chứa Sidebar, Topbar và PlayerBar (không bao giờ unmount)
- **Route Groups:** (auth) group cho login/register
- **Client Components:** Các component cần tương tác (player, forms) được đánh dấu "use client"

2.3.2. Zustand — State Management

Zustand là thư viện quản lý state nhỏ gọn cho React. EMusic dùng 2 store:

- **authStore:** Quản lý thông tin đăng nhập (user, token), các action login/logout
- **playerStore:** Quản lý trình phát nhạc (bài đang phát, trạng thái play/pause, volume, shuffle, repeat)

Điểm đặc biệt: đối tượng Audio được tạo ở mức module (ngoài React state) để tránh re-render không cần thiết.

2.3.3. Tailwind CSS

Tailwind CSS là framework CSS utility-first cho phép thiết kế trực tiếp trong HTML/JSX. EMusic tùy chỉnh Tailwind với bảng màu sp-* (Spotify-inspired):

- **sp-black (#121212):** Background chính
- **sp-surface (#181818):** Surface cards
- **sp-green (#1ed760):** Màu nhấn Spotify Green — play buttons, active states

2.4. Các công nghệ và thư viện khác

2.4.1. AutoMapper

AutoMapper là thư viện giúp ánh xạ (mapping) tự động giữa các object. EMusic dùng AutoMapper để chuyển đổi giữa Entities và DTOs, tránh lặp code mapping thủ công.

2.4.2. *FluentValidation*

FluentValidation cung cấp cú pháp fluent để định nghĩa validation rules cho DTOs. Tự động tích hợp với ASP.NET Core model validation pipeline.

2.4.3. *BCrypt.Net*

BCrypt.Net được dùng để hash mật khẩu người dùng. BCrypt là thuật toán hashing thích ứng, an toàn hơn MD5/SHA vì có cost factor điều chỉnh được.

2.4.4. *HTTP Range Requests*

HTTP Range Requests (RFC 7233) cho phép client request một phần của resource. Khi `EnableRangeProcessing = true` trong `FileStreamResult`, trình phát nhạc có thể tua đến bất kỳ vị trí nào mà không cần tải toàn bộ file, server trả về HTTP 206 Partial Content.

CHƯƠNG 3: HIỆN THỰC HÓA NGHIÊN CỨU

3.1. Mô tả bài toán

EMusic là nền tảng nghe nhạc trực tuyến với 3 loại người dùng:

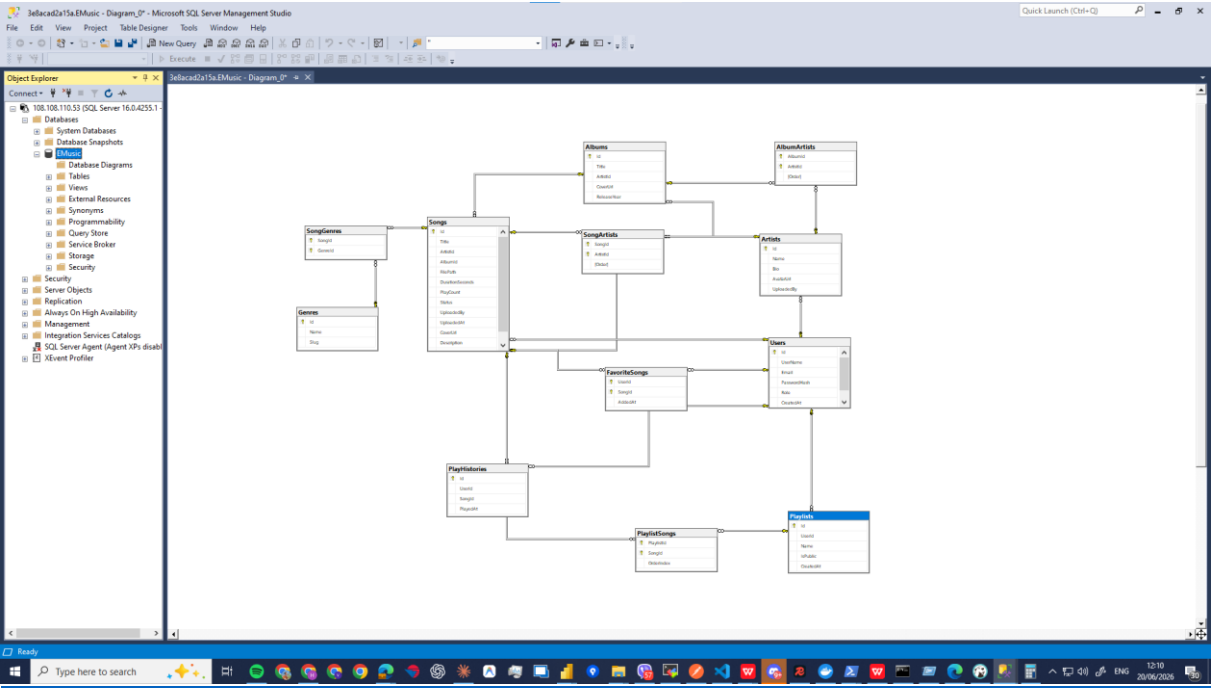
- **User (người dùng thông thường):** Nghe nhạc, tạo playlist, yêu thích bài hát, upload nhạc (cần kiểm duyệt)
- **UserPro (người dùng Pro):** Tương tự User nhưng bài upload được phê duyệt tự động
- **Admin:** Toàn quyền quản lý hệ thống, duyệt/từ chối bài hát

3.2. Yêu cầu chức năng

Mã	Chức năng	Người dùng
UC01	Đăng ký tài khoản	Khách
UC02	Đăng nhập / Đăng xuất	Tất cả
UC03	Nghe nhạc trực tuyến	Tất cả
UC04	Tìm kiếm bài hát	Tất cả
UC05	Quản lý playlist	User, UserPro, Admin
UC06	Yêu thích bài hát	User, UserPro, Admin
UC07	Upload bài hát	User, UserPro, Admin
UC08	Xem lịch sử nghe	User, UserPro, Admin
UC09	Duyệt bài hát	Admin
UC10	Quản lý người dùng	Admin
UC11	Xem thống kê hệ thống	Admin

Bảng 1 Chức năng hệ thống

3.3. Mô hình cơ sở dữ liệu



Hình 1 Mô hình cơ sở dữ liệu (ERD)

Hệ thống EMusic sử dụng SQL Server với 12 bảng dữ liệu. Sau đây là mô tả các bảng chính:

3.3.1 Mô tả bảng Users

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
Id	int	PRIMARY KEY, IDENTITY	Khóa chính
UserName	nvarchar(100)	NOT NULL, UNIQUE	Tên đăng nhập
Email	nvarchar(255)	NOT NULL, UNIQUE	Email
PasswordHash	nvarchar(500)	NOT NULL	Mật khẩu đã hash BCrypt
Role	nvarchar(50)	NOT NULL, DEFAULT "User"	Quyền: User/UserPro/Admin
CreatedAt	datetime2	NOT NULL	Thời gian tạo

Bảng 2 Mô tả bảng Users

3.3.2 Mô tả bảng Songs

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
Id	int	PRIMARY KEY, IDENTITY	Khóa chính
Title	nvarchar(255)	NOT NULL	Tên bài hát
ArtistId	int	FOREIGN KEY → Artists	Nghệ sĩ chính
AlbumId	int	FOREIGN KEY → Albums, NULL	Album (tùy chọn)
FilePath	nvarchar(500)	NOT NULL	Đường dẫn file âm thanh
CoverUrl	nvarchar(500)	NULL	Ảnh bìa
DurationSeconds	int	NOT NULL	Thời lượng (giây)
PlayCount	int	DEFAULT 0	Số lượt nghe
Status	nvarchar(50)	NOT NULL	Pending/Approved/Rejected
UploadedBy	int	FOREIGN KEY → Users	Người upload
UploadedAt	datetime2	NOT NULL	Thời gian upload

Bảng 3 Mô tả bảng Songs

3.3.3 Mô tả bảng Artists

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
Id	int	PRIMARY KEY, IDENTITY	Khóa chính
Name	nvarchar(255)	NOT NULL	Tên nghệ sĩ
Bio	nvarchar(2000)	NULL	Tiểu sử
AvatarUrl	nvarchar(500)	NULL	Ảnh đại diện
UploadedBy	int	FOREIGN KEY → Users	Người tạo
CreatedAt	datetime2	NOT NULL	Thời gian tạo

Bảng 4 Mô tả bảng Artists

3.3.4 Mô tả bảng Albums

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
Id	int	PRIMARY KEY, IDENTITY	Khóa chính
Title	nvarchar(255)	NOT NULL	Tên album
ArtistId	int	FOREIGN KEY → Artists	Nghệ sĩ
CoverUrl	nvarchar(500)	NULL	Ảnh bìa
ReleaseYear	int	NULL	Năm phát hành

Bảng 5 Mô tả bảng Albums

3.3.5 Mô tả bảng Playlists

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
Id	int	PRIMARY KEY, IDENTITY	Khóa chính
Name	nvarchar(255)	NOT NULL	Tên playlist
UserId	int	FOREIGN KEY → Users	Chủ sở hữu
IsPublic	bit	DEFAULT 0	Công khai hay riêng tư
CreatedAt	datetime2	NOT NULL	Thời gian tạo

Bảng 6 Mô tả bảng Playlists

3.3.6 Mô tả bảng PlaylistSongs

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
PlaylistId	int	PK + FK → Playlists	Khóa ngoại
SongId	int	PK + FK → Songs	Khóa ngoại
OrderIndex	int	NOT NULL	Thứ tự trong playlist
AddedAt	datetime2	NOT NULL	Thời gian thêm

Bảng 7 Mô tả bảng PlaylistSongs

3.3.7 Mô tả bảng *FavoriteSongs*

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
UserId	int	PK + FK → Users	Khóa ngoại
SongId	int	PK + FK → Songs	Khóa ngoại
AddedAt	datetime2	NOT NULL	Thời gian yêu thích

Bảng 8 Mô tả bảng FavoriteSongs

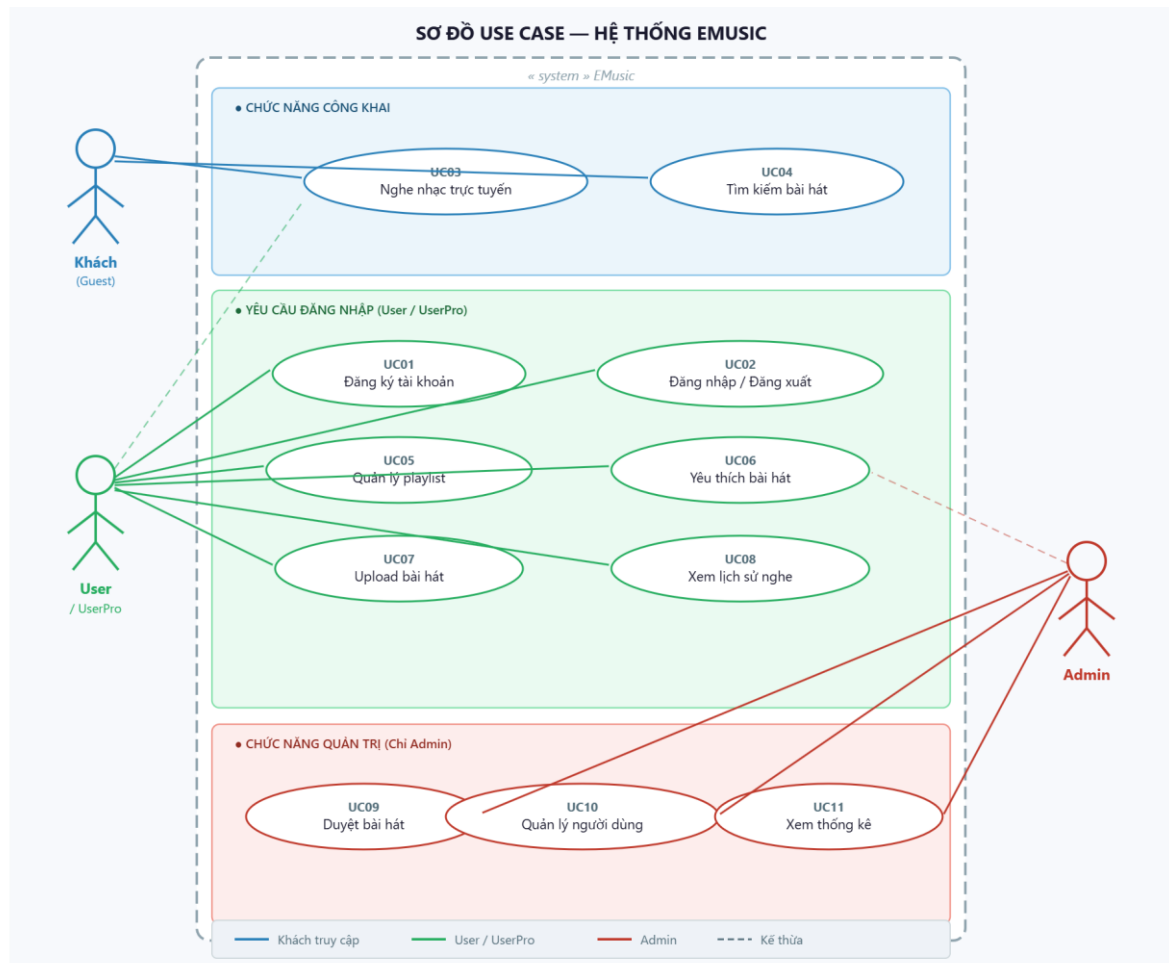
3.3.8 Mô tả bảng *PlayHistories*

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
Id	int	PRIMARY KEY, IDENTITY	Khóa chính
UserId	int	FOREIGN KEY → Users	Người dùng
SongId	int	FOREIGN KEY → Songs	Bài hát
PlayedAt	datetime2	NOT NULL	Thời điểm nghe

Bảng 9 Mô tả bảng PlayHistories

Ngoài ra còn các bảng phụ: Genres, SongGenres (n-n giữa Song và Genre), SongArtists (n-n giữa Song và Artist), AlbumArtists (n-n giữa Album và Artist).

3.4. Sơ đồ Use Case

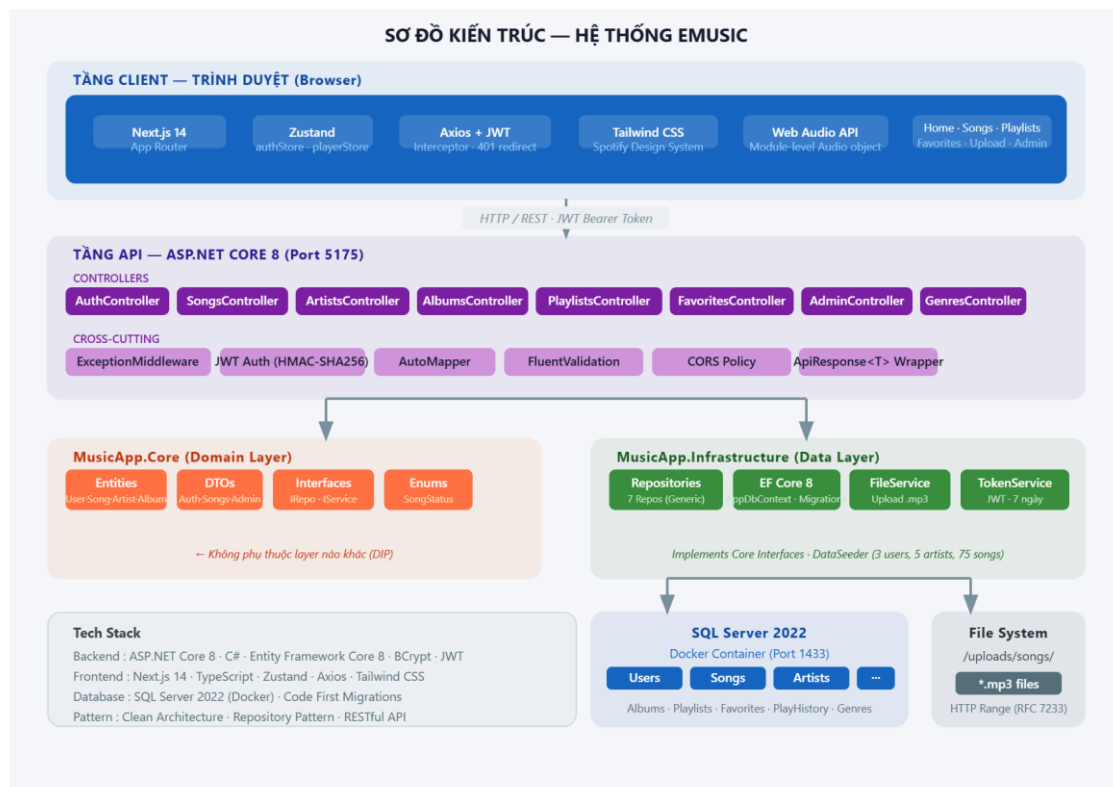


Hình 2 Sơ đồ Use Case

Sơ đồ Use Case thể hiện 3 nhóm Actor chính trong hệ thống EMusic:

- **Khách (Guest):** Chỉ xem và nghe nhạc, không cần đăng nhập
- **Người dùng đăng nhập (User/UserPro):** Thêm chức năng playlist, yêu thích, upload
- **Admin:** Toàn quyền quản lý hệ thống bao gồm duyệt bài, quản lý người dùng, xem thống kê

3.5. Kiến trúc hệ thống



Hình 3 Kiến trúc hệ thống

Hệ thống EMusic được chia thành các thành phần:

Backend (ASP.NET Core 8):

- **API Layer:** 8 Controllers xử lý HTTP requests
- **Core Layer:** Business logic, Entities, Interfaces
- **Infrastructure Layer:** EF Core, Repositories, Services
- **SQL Server 2022:** Cơ sở dữ liệu (chạy Docker)
- **File System:** Lưu trữ file âm thanh và ảnh

Frontend (Next.js 14):

- **App Router:** Routing và layouts
- **Zustand Stores:** Quản lý state
- **Axios:** HTTP client với JWT interceptor
- **Tailwind CSS:** Styling

Luồng xử lý request:

1. Frontend gửi HTTP request đến Backend API
2. ExceptionMiddleware xử lý lỗi toàn cục
3. Authentication/Authorization kiểm tra JWT token
4. Controller nhận request, gọi Repository
5. Repository thao tác với Database qua EF Core
6. Response trả về DTO qua AutoMapper

3.6. Thiết kế API

EMusic thiết kế theo chuẩn RESTful với các nguyên tắc:

- URI dùng danh từ số nhiều: /api/songs, /api/artists
- HTTP methods: GET (đọc), POST (tạo), PUT (cập nhật), PATCH (cập nhật một phần), DELETE (xóa)
- Response nhất quán với ApiResponse<T> wrapper
- Pagination với PagedResult<T>
- HTTP status codes chuẩn: 200, 201, 204, 400, 401, 403, 404, 500

3.6.1 Danh sách API Endpoints chính

Method	Endpoint	Mô tả	Auth
POST	/api/auth/register	Đăng ký	Public
POST	/api/auth/login	Đăng nhập	Public
GET	/api/auth/me	Thông tin user hiện tại	JWT
GET	/api/songs	Danh sách bài hát (paging+search)	Public
GET	/api/songs/{id}/stream	Stream audio	Public
POST	/api/songs/{id}/play	Ghi nhận lượt nghe	Public

Method	Endpoint	Mô tả	Auth
POST	/api/songs	Upload bài hát	JWT
DELETE	/api/songs/{id}	Xóa bài hát	JWT (owner/admin)
GET	/api/artists	Danh sách nghệ sĩ	Public
GET	/api/albums/{id}	Chi tiết album	Public
GET	/api/playlists	Danh sách playlist	JWT
POST	/api/playlists	Tạo playlist	JWT
POST	/api/playlists/{id}/songs	Thêm bài vào playlist	JWT
DELETE	/api/playlists/{id}/songs/{songId}	Xóa bài khỏi playlist	JWT
POST	/api/favorites/{songId}	Toggle yêu thích	JWT
GET	/api/admin/stats	Thống kê tổng quan	Admin
GET	/api/admin/songs/pending	Bài hát chờ duyệt	Admin
PATCH	/api/admin/songs/{id}/approve	Duyệt bài hát	Admin
PATCH	/api/admin/songs/{id}/reject	Từ chối bài hát	Admin

Bảng 10 Danh sách API Endpoints chính

3.6.2 Tài khoản mẫu được seed sẵn

Role	Email	Password	Mô tả
Admin	170124891@rdi.edu.vn	Admin@12345	Quản trị viên
UserPro	hieuviet.itc@gmail.com	Pro@12345	Upload tự động duyệt
User	hieuviet.1103@gmail.com	User@12345	Upload cần kiểm duyệt

Bảng 11 Tài khoản mẫu được seed sẵn

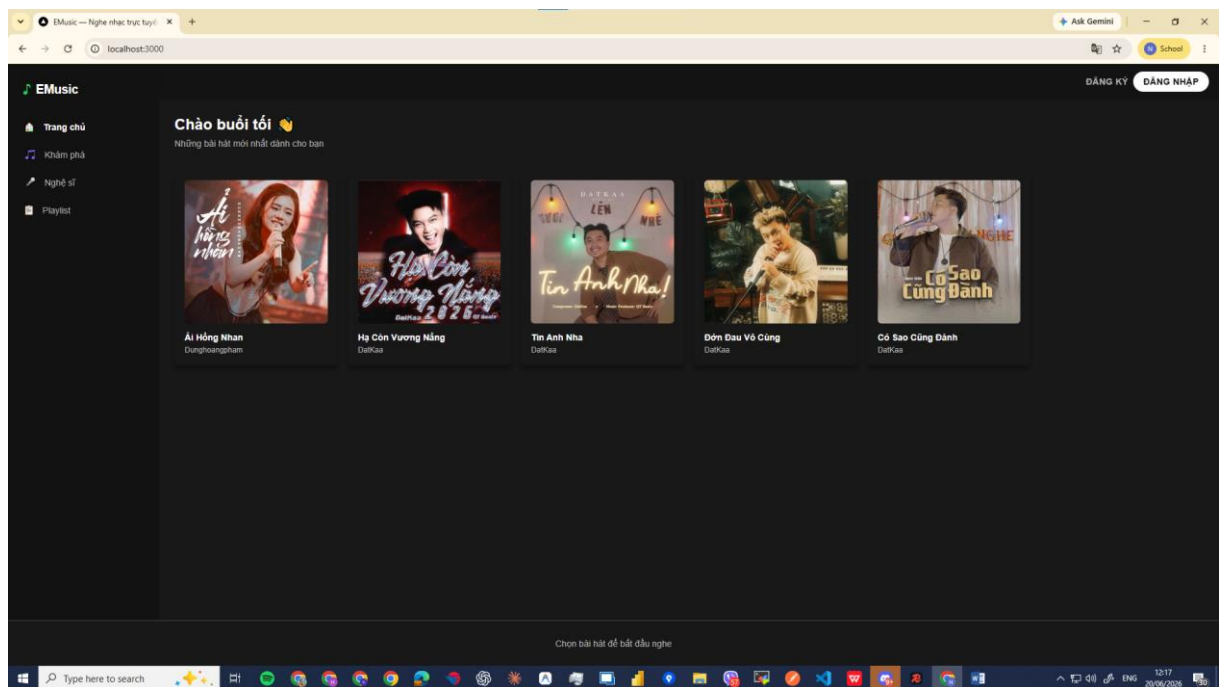
CHƯƠNG 4: KẾT QUẢ NGHIÊN CỨU

4.1. Môi trường chạy ứng dụng

Ứng dụng EMusic chạy với cấu hình:

- **Backend:** <http://localhost:5175> (ASP.NET Core 8)
- **Frontend:** <http://localhost:3000> (Next.js 14)
- **Database:** SQL Server 2022 trên Docker (port 1433)
- **Swagger UI:** <http://localhost:5175/swagger>

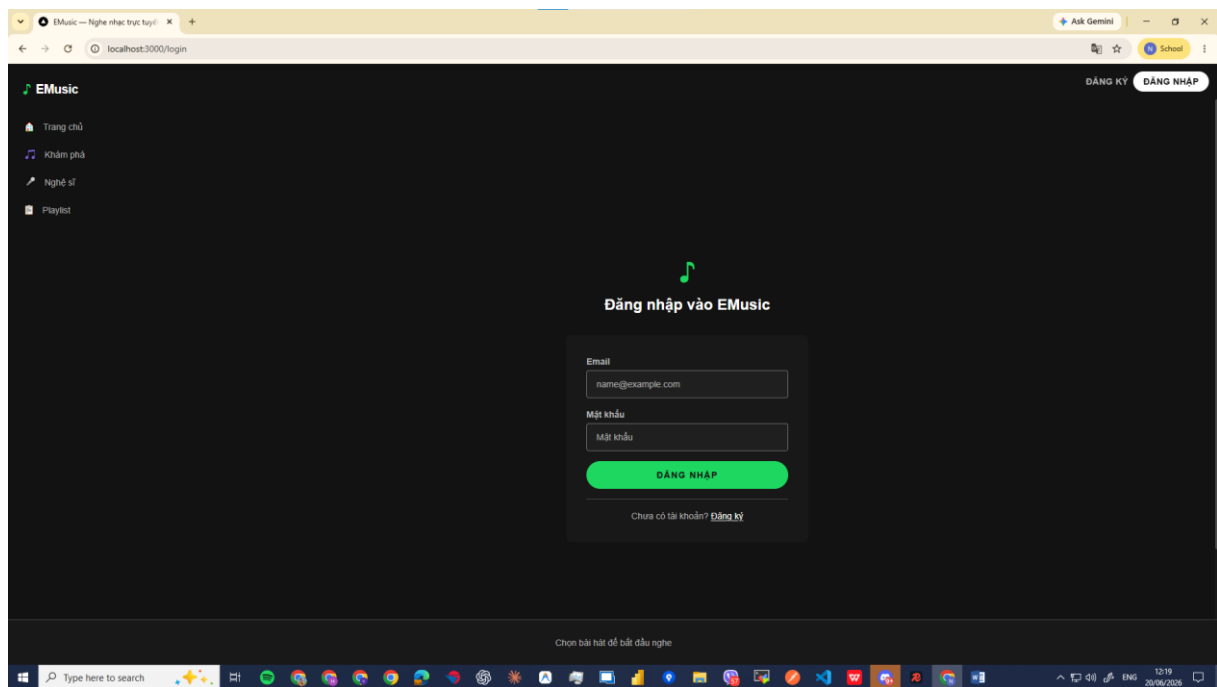
4.2. Giao diện trang chủ



Hình 4 Giao diện trang chủ

Trang chủ hiển thị các bài hát mới nhất theo dạng card grid với ảnh bìa, tên bài hát và nghệ sĩ. Sidebar bên trái chứa navigation (Trang chủ, Khám phá, Nghệ sĩ, Playlist). Topbar bên phải có nút Đăng ký / Đăng nhập. Khi đăng nhập, sidebar hiện thêm Yêu thích, Bài đã upload và Admin (nếu là admin). Phía dưới là PlayerBar toàn cục.

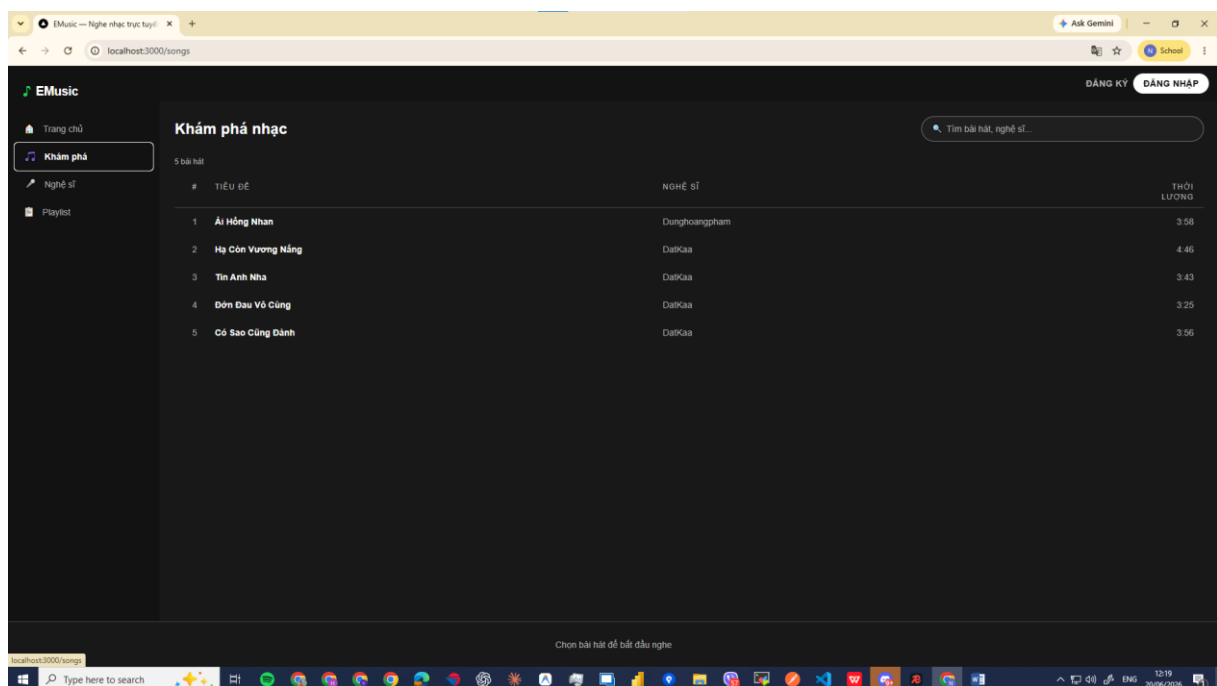
4.3. Giao diện Đăng nhập / Đăng ký



Hình 5 Giao diện trang đăng nhập

Form đăng nhập sử dụng Email và Mật khẩu. Nút ĐĂNG NHẬP màu xanh Spotify Green (#1ed760) theo đúng design system. Sau khi đăng nhập thành công, JWT token được lưu vào localStorage và Zustand store cập nhật trạng thái người dùng.

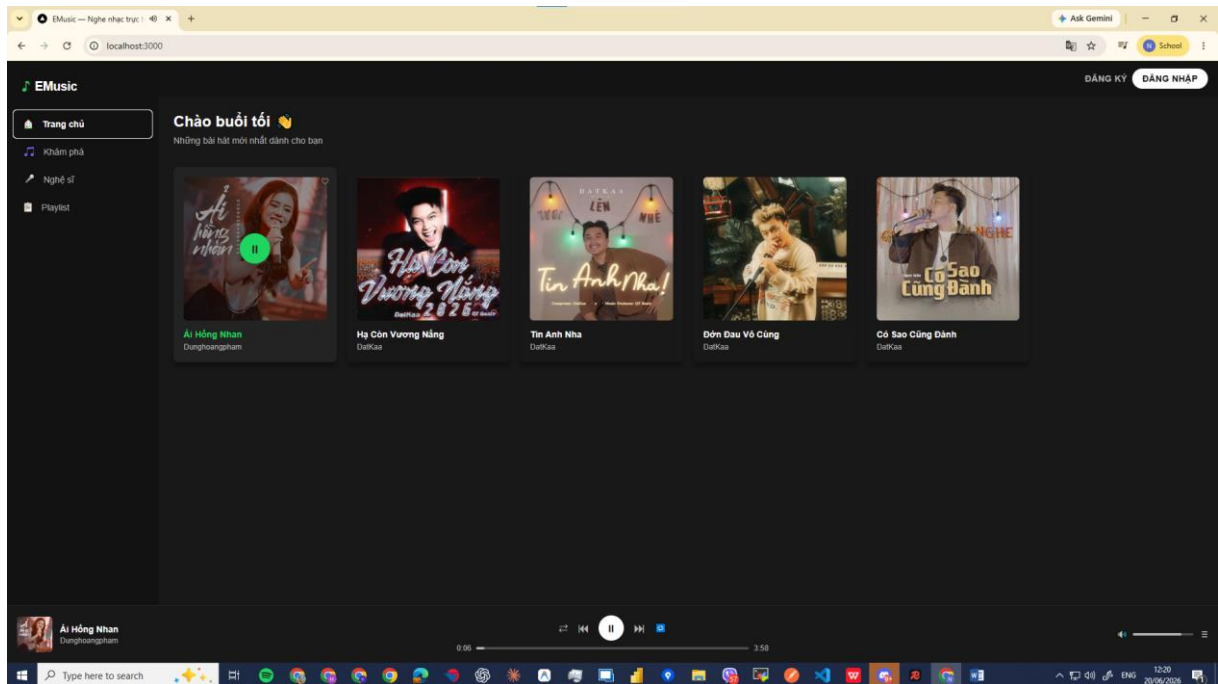
4.4. Giao diện Khám phá nhạc



Hình 6 Giao diện Khám phá nhạc

Trang /songs hiển thị danh sách bài hát dạng bảng với các cột: #, Tiêu đề, Nghệ sĩ, icon Yêu thích, Thời lượng. Ô tìm kiếm ở góc trên phải hỗ trợ debounce 300ms. Nhấn vào bất kỳ hàng nào để phát nhạc.

4.5. Trình phát nhạc (PlayerBar)

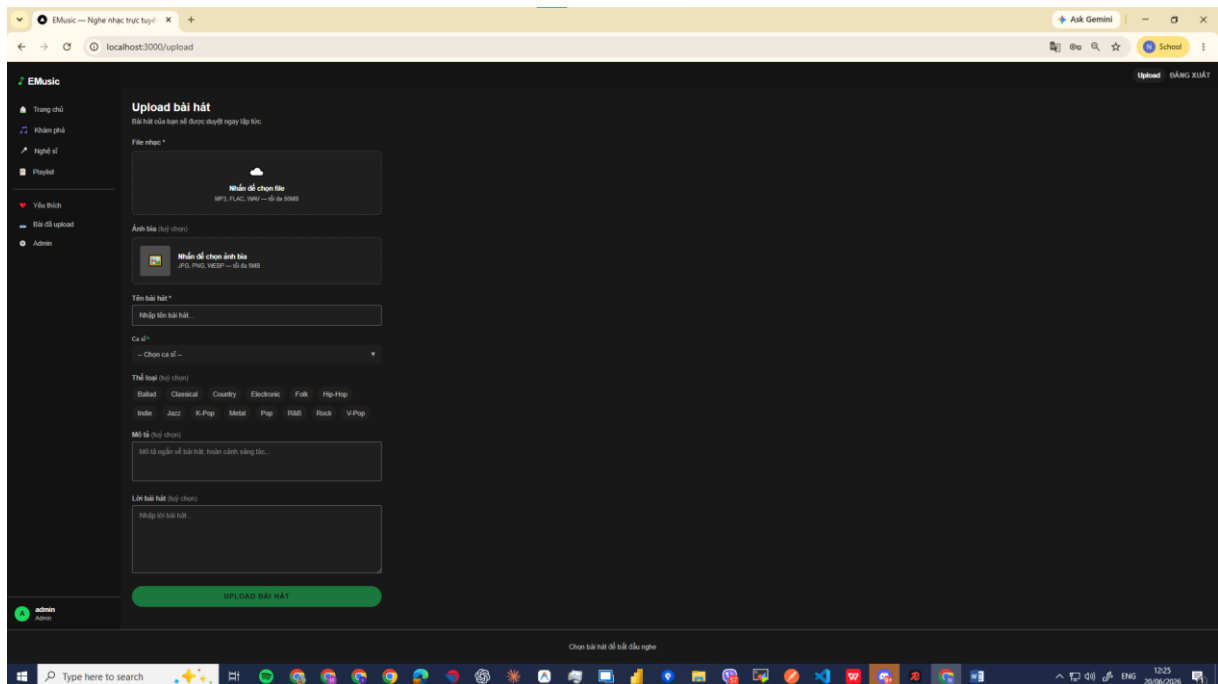


Hình 7 Trình phát nhạc (PlayerBar)

PlayerBar cố định ở dưới cùng trang, không bao giờ bị unmount khi chuyển trang (đặt trong root layout). Bao gồm:

- Thông tin bài đang phát (tên, nghệ sĩ)
- Các nút điều khiển: Shuffle, Previous, Play/Pause, Next, Repeat
- Progress bar với thời gian đã phát / tổng thời gian (có thể click để tua)
- Thanh điều chỉnh âm lượng

4.6. Giao diện Upload bài hát

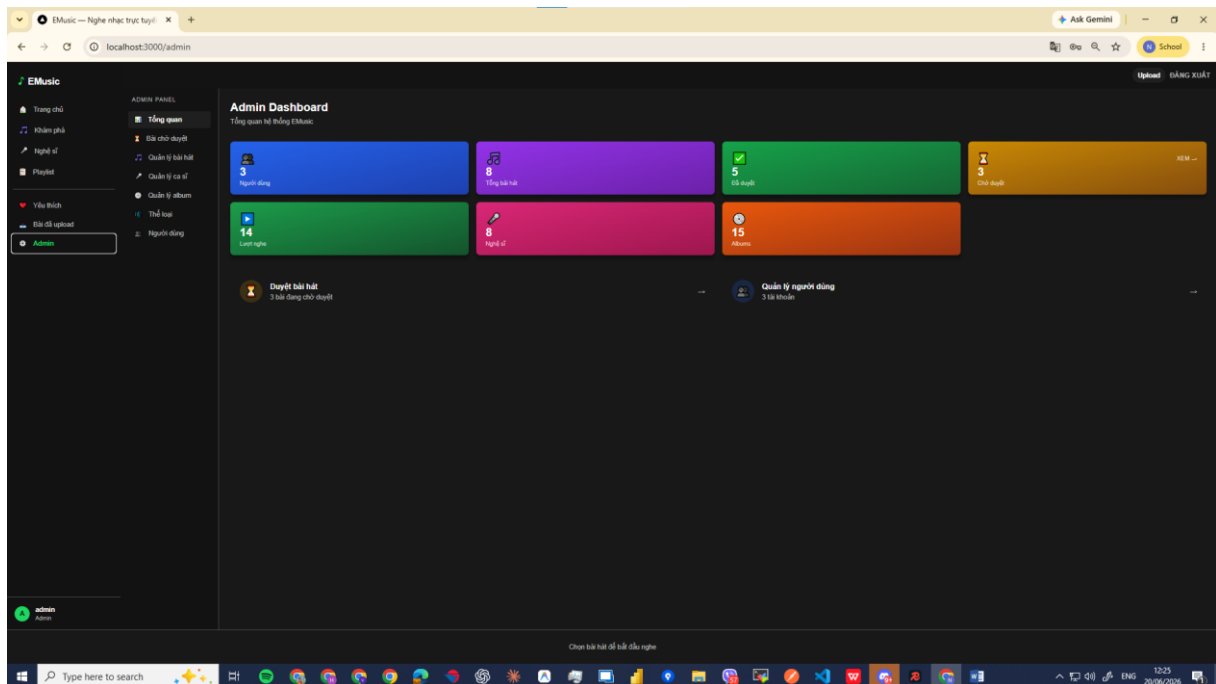


Hình 8 Giao diện upload bài hát

Trang /upload cho phép người dùng đã đăng nhập upload file nhạc (.mp3, .flac, .wav). Web Audio API tự động detect thời lượng bài hát. Trạng thái bài sau upload phụ thuộc vào role:

- **Admin/UserPro:** Approved ngay (hiển thị "Đã đăng")
- **User:** Pending (hiển thị "Đang chờ duyệt")

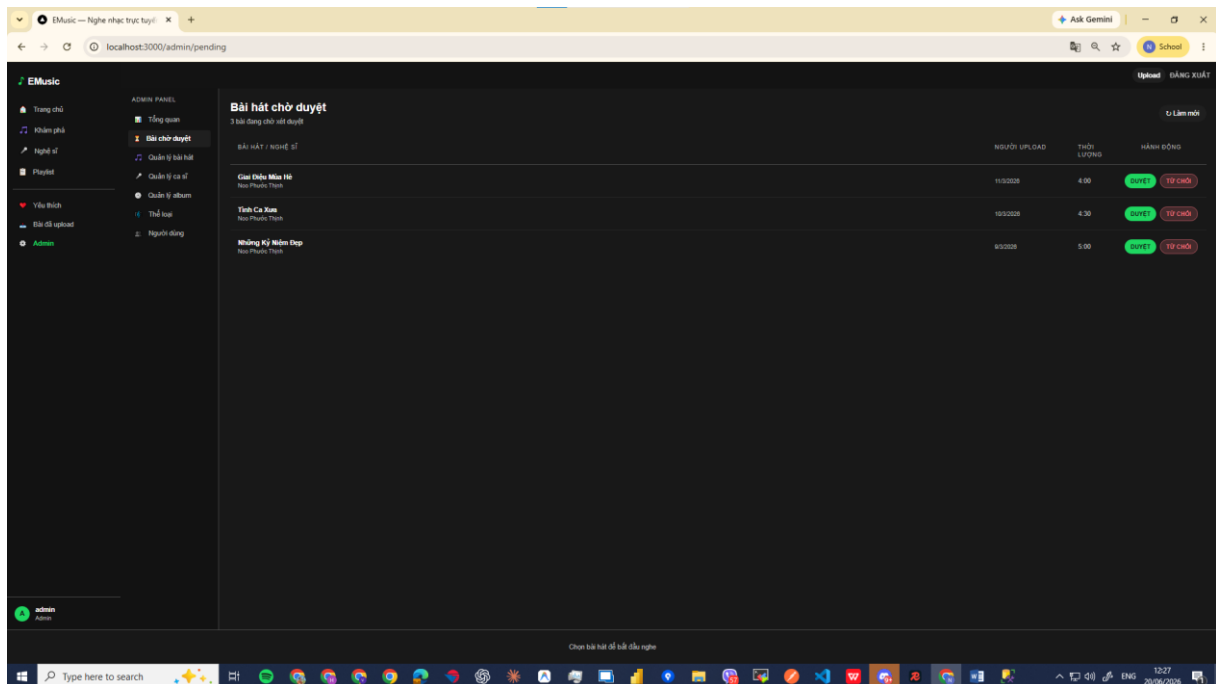
4.7. Admin Dashboard



Hình 9 Admin Dashboard

Trang /admin (chỉ Admin mới truy cập được) hiển thị 7 thống kê tổng quan hệ thống với màu sắc phân biệt theo loại thông tin. Quick links dẫn đến các trang quản lý con. Layout admin có sidebar riêng với các mục: Tổng quan, Bài chờ duyệt, Quản lý bài hát, Quản lý ca sĩ, Quản lý album, Thể loại, Người dùng.

4.8. Quản lý bài hát chờ duyệt

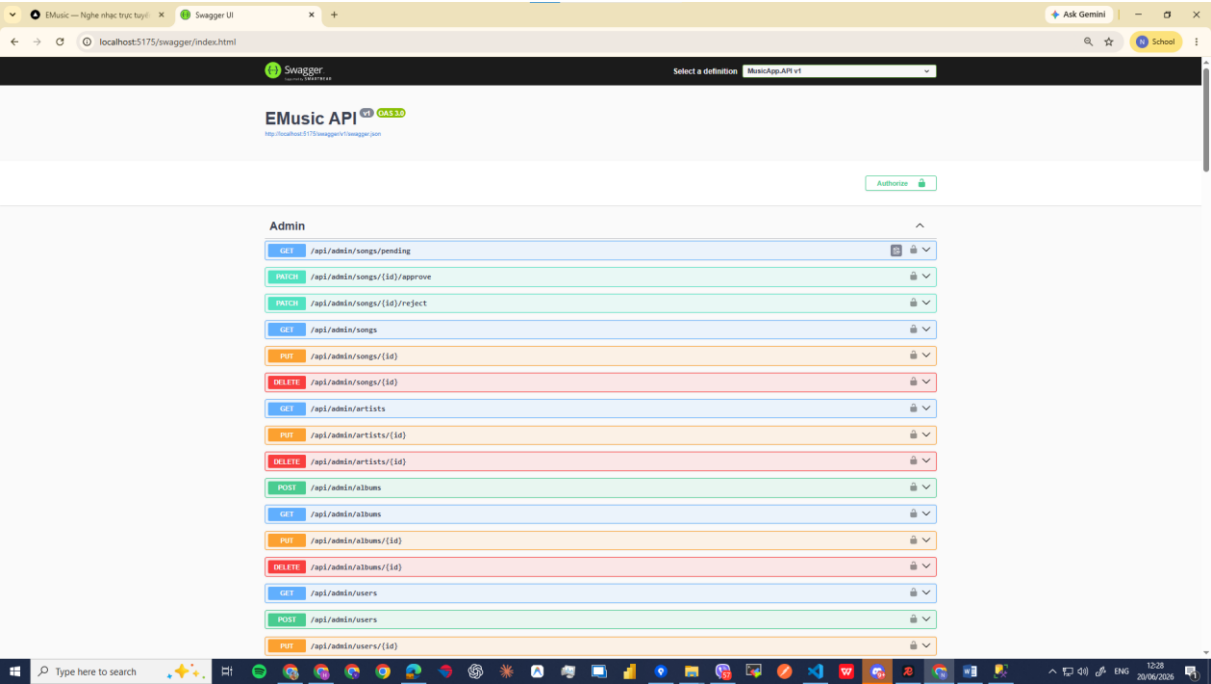


Hình 10 Quản lý bài hát chờ duyệt

Trang /admin/pending liệt kê các bài hát đang ở trạng thái Pending. Admin có thể:

- Nhấn "DUYỆT" để chuyển trạng thái Pending → Approved (bài sẽ xuất hiện trên trang công khai)
- Nhấn "TỪ CHỐI" để chuyển Pending → Rejected
- UI cập nhật optimistically (row biến mất ngay khi nhấn)

4.9. Swagger API Documentation



Hình 11 Swagger API Documentation

Swagger UI tại /swagger cung cấp tài liệu API đầy đủ với khả năng test trực tiếp. Hỗ trợ xác thực JWT thông qua nút Authorize. Các endpoint được nhóm theo controller: Admin, Albums, Artists, Auth, Favorites, Genres, Playlists, Songs.

4.10. Đánh giá kết quả

Sau quá trình thực hiện, EMusic đã hoàn thành các mục tiêu đề ra:

Mục tiêu	Kết quả
Backend ASP.NET Core 8 với Clean Architecture	Hoàn thành — 3 projects, 8 controllers, 35+ endpoints
RESTful API với JWT Authentication	Hoàn thành — Swagger UI đầy đủ
Phân quyền 3 cấp (User/UserPro/Admin)	Hoàn thành
Stream audio với HTTP Range Request	Hoàn thành — Hỗ trợ seek, trả 206 Partial Content
Frontend Next.js với Spotify Design	Hoàn thành — 10 trang, dark theme
Quản lý playlist	Hoàn thành — CRUD + reorder

Mục tiêu	Kết quả
Admin Dashboard	Hoàn thành — 7 thống kê, duyệt bài
Docker cho SQL Server	Hoàn thành

Bảng 12 Đánh giá kết quả

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Đồ án đã xây dựng thành công website nghe nhạc trực tuyến EMusic với đầy đủ các chức năng cốt lõi của một nền tảng streaming hiện đại. Qua quá trình thực hiện, đồ án đã đạt được các kết quả sau:

Về kỹ thuật backend, hệ thống được xây dựng theo kiến trúc Clean Architecture rõ ràng, giúp code dễ bảo trì và mở rộng. REST API được thiết kế đúng chuẩn với xác thực JWT, phân quyền theo role và xử lý lỗi tập trung qua Middleware. Tính năng stream audio với HTTP Range Request đảm bảo trải nghiệm nghe nhạc mượt mà, hỗ trợ tua và phát từ bất kỳ vị trí nào.

Về giao diện, frontend Next.js mang lại trải nghiệm người dùng hiện đại với dark theme theo phong cách Spotify, PlayerBar toàn cục không bị gián đoạn khi điều hướng giữa các trang, và tìm kiếm thời gian thực với debounce.

Về công nghệ, đồ án đã áp dụng thành công nhiều công nghệ và thư viện quan trọng trong hệ sinh thái .NET và JavaScript hiện đại: Entity Framework Core, AutoMapper, FluentValidation, Zustand, Tailwind CSS, Docker.

5.2. Hướng phát triển

Trong tương lai, EMusic có thể được mở rộng theo các hướng:

- 1. Triển khai trên Cloud:** Deploy backend lên Azure App Service hoặc AWS EC2, frontend lên Vercel, database lên Azure SQL Database
- 2. Tính năng xã hội:** Cho phép người dùng theo dõi nhau, chia sẻ playlist, comment và like bài hát
- 3. Đề xuất bài hát (Recommendation):** Áp dụng Machine Learning để gợi ý bài hát dựa trên lịch sử nghe
- 4. Tài khoản Premium:** Bổ sung hệ thống subscription với các quyền lợi như không quảng cáo, tải nhạc offline

- 5. **Mobile App:** Phát triển ứng dụng di động với React Native hoặc Flutter sử dụng chung backend API
- 6. **Tối ưu hiệu năng:** CDN cho file âm thanh, caching Redis, lazy loading album art
- 7. **Lyrics thời gian thực:** Hiển thị lời bài hát đồng bộ với tiến trình phát nhạc (LRC format)

DANH MỤC TÀI LIỆU THAM KHẢO

- [1] Microsoft, "ASP.NET Core documentation," Microsoft Docs, 2024. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core>
- [2] Microsoft, "Entity Framework Core documentation," Microsoft Docs, 2024. [Online]. Available: <https://learn.microsoft.com/en-us/ef/core>
- [3] Next.js Team, "Next.js Documentation," Vercel, 2024. [Online]. Available: <https://nextjs.org/docs>
- [4] R. C. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall, 2017.
- [5] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, May 2015.
- [6] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, University of California, Irvine, 2000.
- [7] Tailwind Labs, "Tailwind CSS Documentation," 2024. [Online]. Available: <https://tailwindcss.com/docs>
- [8] Docker Inc., "Docker Documentation," 2024. [Online]. Available: <https://docs.docker.com>
- [9] PMJ Software, "AutoMapper Documentation," 2024. [Online]. Available: <https://docs.automapper.org>
- [10] Jeremy Skinner, "FluentValidation Documentation," 2024. [Online]. Available: <https://docs.fluentvalidation.net>

PHỤ LỤC

Phụ lục A: Hướng dẫn cài đặt và chạy ứng dụng

Yêu cầu môi trường:

- .NET SDK 8.x
- Node.js 18+
- Docker Desktop
- dotnet-ef tool

Bước 1 — Cài đặt dotnet-ef:

```
dotnet tool install --global dotnet-ef
```

Bước 2 — Khởi động Backend:

```
cd E:\DoAn\...\EMusic  
  
docker compose up -d  
  
dotnet run --project MusicApp.API
```

Bước 3 — Khởi động Frontend:

```
cd music-frontend  
  
npm install  
  
npm run dev
```

Địa chỉ truy cập:

- **Frontend:** <http://localhost:3000>
- **Swagger UI:** <http://localhost:5175/swagger>

Phụ lục B: Cấu trúc thư mục dự án

```

EMusic/
├── MusicApp.sln
├── docker-compose.yml
├── MusicApp.API/
│   ├── Controllers/
│   ├── Mappings/
│   ├── Middleware/
│   ├── Program.cs
│   └── appsettings.json
├── MusicApp.Core/
│   ├── Entities/
│   ├── DTOs/
│   ├── Enums/
│   └── Interfaces/
├── MusicApp.Infrastructure/
│   ├── Data/
│   ├── Migrations/
│   ├── Repositories/
│   └── Services/
└── music-frontend/
    ├── src/app/
    ├── src/components/
    ├── src/store/
    └── src/lib/
    
```